

Code-Sentinel

A Multi-Agent Code Review Platform Built on Service-Oriented Architecture

1. Project Description

CodeSentinel is a code review platform which divides the review work among five independent agent services - Security, Style, Performance, Logic and Documentation. Each agent is a separate web service with its own REST and JSON API, and a central orchestrator built using LangGraph calls all of them in parallel and joins their output into one single report. The same system works from three places - a GitHub App which reviews pull requests automatically, a VS Code extension for review on demand, and a Next.js dashboard. Style and lint problems are fixed automatically using existing tools, while security, logic and performance problems are given as AI suggestions which the developer can accept or reject.

2. Motivation and Problem Statement

In almost every team, code review is the main quality gate before code goes to production, but the whole thing depends on one thing only, that is the free time of a senior developer. Because of this a pull request keeps waiting in the queue for hours or sometimes days. While studying how review actually happens in real teams and in our own college projects, we noticed a few problems. The situation has become worse recently, because AI coding assistants have increased the amount of code being written, but the number of reviewers has not increased:

- Reviewer time goes in the wrong place. Most of the review comments are about naming, formatting, unused imports and missing null checks, and very little time is left for design and business logic, which is where a human is actually needed.
- The quality of review is not consistent. The same code given to two reviewers gives two different sets of comments, and it also depends on how busy the reviewer is on that day.
- Serious issues slip through. In a diff of a few hundred lines it is easy to miss a hardcoded secret or an unsafe database query, and such issues cost much more when they are found later in production.
- Many developers have no reviewer at all. Students, solo developers and small open source projects mostly end up merging their own code without any second opinion.
- The existing tools are scattered. A linter for style, a separate scanner for security, a separate profiler for performance - each one giving its own report in its own format. Nobody has the patience to open four dashboards for one pull request.

Problem Statement

A developer today does not have one single, affordable and extensible system which checks a piece of code for security, style, performance, logic and documentation together, and gives the result at the place where the developer is already working, that is the editor or the pull request. The available tools either check only one of these things, or they come as one closed monolithic product in which a team can neither use a single part alone nor add a check of their own. CodeSentinel aims to fill this gap by giving one combined and automated first-level review, while keeping the developer in full control of what actually gets changed in the code.

Since these five kinds of checks are very different from each other in nature, we are building each one as a separate service which can run, scale and be reused on its own. This also gives us a practical way of applying SOA concepts in a real working project instead of only theory.

3. Application Type

CodeSentinel is a hybrid application. It is not one single app, it is a set of REST based backend services used by three different clients.

- **Web (admin side)** - Next.js dashboard for review history, reports and configuration. It will be mobile responsive so that the status can be checked from a phone browser also.
- **Client side** - VS Code Extension and GitHub App. For a code review tool the developer works inside the editor and on GitHub, so these act as the client side applications in place of a mobile app.

- **Backend** - REST and JSON web services in Node.js and Python, so any new client, including a mobile app, can be added later without changing any service.

4. Target Users and Use Cases

- Students and solo developers who have nobody available to review their code.
- Small development teams where every pull request waits for one busy senior developer.
- Open source maintainers who receive contributions from unknown contributors and need a first level check.
- Tech leads who want to spend their review time on design instead of formatting.
- Repository admins who configure which agents run for which repository and see the reports.

5. Scope and High-Level Features

- One combined review report covering security, style, performance, logic and documentation.
- Three entry points - GitHub pull requests (webhook based), VS Code extension (on demand) and the web dashboard.
- Style and lint issues fixed automatically using ESLint, Prettier, Ruff and Black. No LLM is needed for these, so the fixes are safe and predictable.
- Security, logic and performance issues given as AI suggestions for high-confidence cases only, posted as GitHub suggested changes. The developer applies them, nothing is changed automatically.
- Agents called in parallel with a timeout, so a slow agent returns partial results instead of blocking.
- Context-aware review using a vector database which brings up similar issues found earlier.
- Out of scope for now - languages other than JavaScript/TypeScript and Python, IDEs other than VS Code, automatic merging of pull requests, and a dedicated mobile application.

6. System Architecture

The system has five layers. The three clients talk only to the API Gateway, which passes the request to the LangGraph orchestrator. The orchestrator calls the five agent services in parallel over REST and JSON, and the agents use PostgreSQL and the vector database. No agent calls another agent directly, so each one can be developed, deployed, scaled and even reused outside CodeSentinel on its own. This is where the SOA principles of service decomposition, orchestration, loose coupling, interoperability, reusability and independent scalability come into the picture.

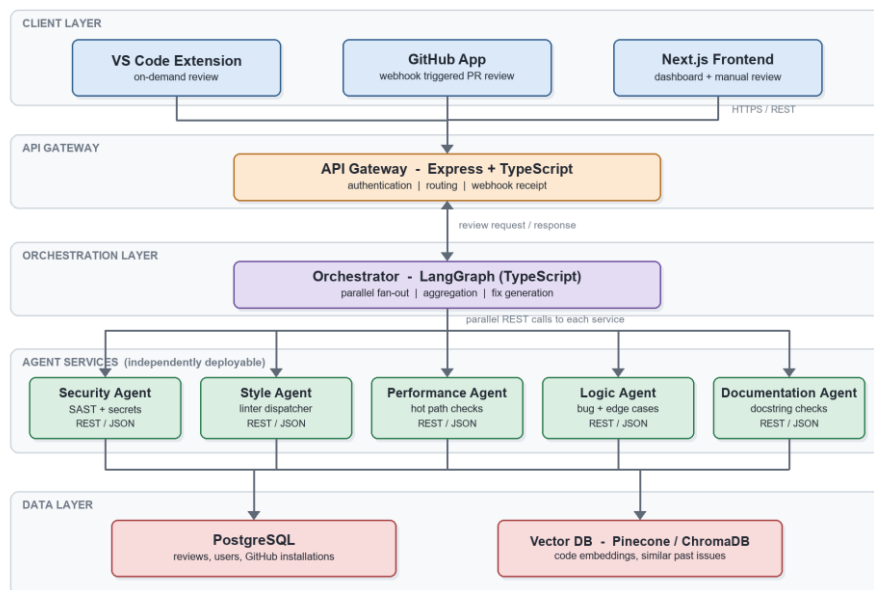


Figure 1: CodeSentinel layered system architecture

7. Technology Stack

Area	Technology and purpose
Frontend	Next.js - dashboard, review history, reports and manual code submission
Gateway	Express + TypeScript - single entry point, authentication, routing, webhook receipt
Orchestration	LangGraph (TypeScript) - parallel fan-out to agents, aggregation, fix generation
Agent services	Node.js / Python REST services, one per review type. Linters (ESLint, Prettier, Ruff, Black) run inside a Docker sandbox
Database	PostgreSQL - reviews, users and GitHub installation details
Vector database	Pinecone / ChromaDB - code embeddings and retrieval of similar past issues
LLM layer	Provider-agnostic interface over OpenRouter, Gemini Flash and Groq, with local Ollama for development
Editor and VCS	VS Code Extension API; GitHub App using Octokit, Webhooks and Checks API
Tooling	Turborepo Monorepo, Docker for deployment and CI

8. Team Members, Roles and Responsibilities

S.No	Name	Enrollment No.	Role and Responsibilities	Hours
1	Sumit Goyal	202512092	Orchestrator (LangGraph) - parallel fan-out and timeout handling. LLM abstraction layer.	200
2	Durgesh	202512051	Orchestrator - result aggregation, duplicate removal, severity ranking. Vector DB embeddings and similarity search.	200
3	Tej Prakash Tak	202512113	Security Agent service (SAST, secret detection) and Logic Agent service (bugs, edge cases).	200
4	Nevil Nandasana	202512009	Style Agent - language dispatcher, Docker sandbox, linter integration. VS Code extension commands.	200
5	Parin Makwana	202512098	Performance Agent service and Documentation Agent service (docstring suggestions).	200
6	Ayush Soni	202512030	API Gateway (Express + TypeScript) - auth, routing, webhook receipt. PostgreSQL schema.	200
7	Vatsal Kamani	202512068	GitHub App - webhooks, Octokit, Checks and Review API. Docker and CI deployment.	200
8	Moksh Mehta	202512120	Next.js dashboard, review history and reports. VS Code webview panel UI.	200
	Total	8 members	Each member: 140 hrs specialised work + 60 hrs common work (design, testing, documentation)	1600

Responsibilities are not strictly fixed - every area has a second member familiar with it, and the team collaborates across areas during integration and testing.

9. Timeline, Milestones and Effort Distribution

The project is planned for a 14 week semester. Every member puts in around 14 to 15 hours per week, which comes to 200 person-hours per member and 1600 person-hours for the team. The work is done in three phases so that a working system (MVP) is ready by Week 10 and the remaining features are added on top of it.

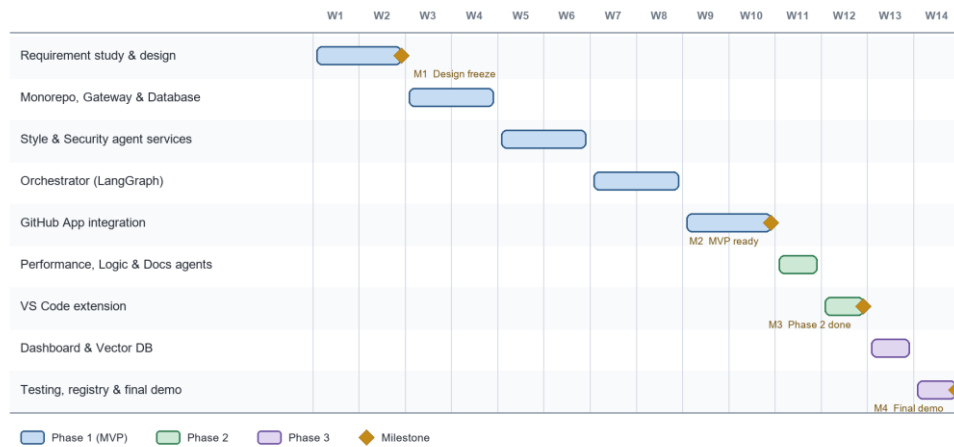


Figure 2: 14 week timeline with phases and milestones (M1 to M4)

Weeks	Work package / tasks	Team member(s)	Hours
W1-W2	Requirement study, service boundaries and API contract design (M1: design freeze)	All 8 members	160
W3-W4	Monorepo setup, API Gateway and PostgreSQL schema	Ayush, Vatsal	180
W5-W6	Style Agent and Security Agent services, Docker sandbox for linters	Nevil, Tej	230
W7-W8	Orchestrator using LangGraph, and the LLM abstraction layer	Sumit, Durgesh	210
W9-W10	GitHub App integration and Vector DB integration (M2: MVP ready)	Vatsal, Durgesh	170
W11	Performance, Logic and Documentation agent services	Parin, Tej	140
W12	VS Code extension - commands and webview panel (M3: Phase 2 done)	Nevil, Moksh	95
W13	Next.js dashboard, reports and review history	Moksh, Parin	95
W5-W14	Integration, testing and bug fixing (runs in parallel with development)	All 8 members	200
W12-W14	Documentation, final report and demonstration (M4: final demo)	All 8 members	120
14 weeks	Total estimated effort	8 members x 200 hrs	1600